

Reflektionsfrågor - Password Test Suite

How many tests are needed to find all bugs in the example?

Svar: 9 tester behövs för att hitta alla 10 buggar.

I den optimerade testsviten har jag:

- **Test 1-6:** Fångar var sin unik bugg (6 tester)
- **Test 7:** Fångar både BugToShortPassword OCH BugWrongMessage (1 test för 2 buggar)
- **Test 8:** Fångar BugVeryShort (1 test)
- **Test 9:** Fångar BugWrongHashingAlgorithm (1 test)
- **Test 11:** Extra validering för `isPasswordSame` (fångar ingen bugg men testar edge case)

Genom att kombinera test 7 med korrekt felmeddelande-kontroll kan jag detektera två buggar med samma testdata (11 tecken). Detta är mer effektivt än att ha separata tester.

What are the missing tests you think would be good to add?

Förslag på ytterligare tester:

1. Boundary testing för längd

- Test med exakt 12 tecken (minsta giltiga längd) med siffra
- Test med mycket långt lösenord (t.ex. 100+ tecken)

2. Number position testing

- Siffra först i lösenordet: "1validPassword"
- Siffra i mitten: "validPass1word"
- Siffra sist: "validPassword1" (detta testas redan)
- Flera siffror: "valid123Password"

3. Whitespace edge cases

- Endast spaces: " "
- Spaces i mitten: "valid Pass1word"
- Tab och newline tecken

4. Multiple Password comparison

- Test att samma lösenord returnerar true vid jämförelse
- Test med tre olika Password-objekt

5. Special characters

- Lösenord med special tecken: "valid@Pass1word!"
- Unicode tecken: "välid1Påsswörd"

6. Hash collision testing

- Mer omfattande test med hundratalstusentals lösenord för att säkerställa inga kollisioner
-

What is good test data for this example and why?

Bra testdata för Password-klassen:

1. Edge Cases (Gränsfall)

- **Tom sträng** (''): Testar minimal input
- **6 tecken** ('a1b2c3'): Just under minsta längd (hittar BugVeryShort)
- **11 tecken** ('0123456789a'): En tecken under korrekt minimum (hittar BugToShortPassword)
- **12 tecken med siffra**: Minsta giltiga lösenord
- **Mycket långt lösenord**: Testar max capacity

2. Representative Data (Representativ data)

- **Giltigt lösenord med siffra** ('validPassword1'): Normal use case
- **Lösenord utan siffra** ('longenoughpassword'): Testar number-validering
- **Lösenord med spaces** (' validPassword1 '): Testar trimming

3. Special Cases

- **Olika lösenord**: Testar jämförelse-funktionalitet
- **Multiple passwords**: Testar hash-unikhet och kollisioner
- **Invalid argument types**: Testar error handling

Varför är detta bra testdata?

- Täcker alla validators (längd, siffror, trimming)
 - Testar både giltiga och ogiltiga inputs
 - Fokuserar på gränsvärden där buggar ofta uppstår
 - Testar edge cases som tom sträng och whitespace
 - Verifierar att olika lösenord ger olika hash-värden
-

Can we and should we test private methods?

Svar: Vi KAN men bör INTE testa privata metoder direkt.

Varför inte?

1. **Inkapsling**: Privata metoder är implementation details som kan ändras
2. **Fragile tests**: Tester mot privata metoder blir beroende av intern implementation
3. **Public API**: Vi bör testa genom den publika API:n (constructor, getPasswordHash, isPasswordSame)
4. **Refactoring**: Om vi testar privata metoder blir det svårt att refaktorera

I Password-exempel

Privata metoder som `#isTooShort()`, `#containsNumber()`, `#simpleHash()` testas **indirekt** genom:

- Constructor-tester (använder alla tre metoder)
- getPasswordHash() tester (testar #simpleHash indirekt)

När det MÖJLIGEN kan vara OK

- Komplex algoritm i privat metod som är kritisk
- Legacy code med mycket logik i privata metoder
- Men då bör man överväga att extrahera till egen klass istället

Bästa praxis: Testa beteende genom publika metoder, inte implementation.

Can all code be covered by code coverage? Why not?

Svar: NEJ, all kod kan inte alltid täckas av code coverage.

Kod som är svår/omöjlig att täcka

1. Error handling för system errors

- Out of memory exceptions
- Disk full errors
- Network failures (om inte mockad)

2. Defensive programming

```
if (impossibleCondition) {  
    throw new Error('This should never happen')  
}
```

3. Dead code

- Kod som aldrig kan nås p.g.a. logik
- Exempel: `if (true) { ... } else { /* unreachable */ }`

4. Platform-specific kod

- OS-specifika grenar som inte körs på test-miljön

5. Getter/Setter i vårt exempel

- `getPasswordHash()` täcks lätt
- Men vissa trivial getters kanske inte testas explicit

I Password-exempel

Täcks:

- Constructor validation
- Hash generation
- Password comparison

- Error cases

Kan vara svårt att täcka:

- Om hash-algoritmen hade edge cases för extremt långa strängar
- Out of memory vid väldigt stora inputs

Praktiskt mål: Sträva efter 80-100% coverage för business logic, men 100% är inte alltid möjligt eller meningsfullt.

What kind of "asserts"/"expects" can be done in your testing framework?

I Jest finns många typer av expects:

1. Equality Matchers

```
expect(value).toBe(expectedValue)           // Exakt likhet (===)
expect(value).toEqual(expectedValue)         // Deep equality för objekt
expect(value).not.toBe(unexpectedValue)      // Negation
```

2. Truthiness

```
expect(value).toBeTruthy()
expect(value).toBeFalsy()
expect(value).toBeNull()
expect(value).toBeUndefined()
expect(value).toBeDefined()
```

3. Numbers

```
expect(value).toBeGreaterThan(3)
expect(value).toBeGreaterThanOrEqual(3.5)
expect(value).toBeLessThan(5)
expect(value).toBeCloseTo(0.3) // För floating point
```

4. Strings

```
expect(string).toMatch(/pattern/)
expect(string).toContain('substring')
```

5. Arrays and Iterables

```
expect(array).toContain(item)
expect(array).toHaveLength(3)
expect(array).toContainEqual(object)
```

6. Exceptions (används mest i vårt Password-projekt)

```
expect(() => new Password('')).toThrow()
expect(() => new Password('')).toThrow('Too short password')
expect(() => new Password('')).toThrow(Error)
expect(() => function()).not.toThrow()
```

7. Objects

```
expect(object).toHaveProperty('key')
expect(object).toMatchObject({key: 'value'})
```

I Password-tester använder jag

- `toBe()` - För hash-jämförelser
- `not.toBe()` - För att verifiera att hash inte är plain text
- `toThrow()` - För validation errors
- `not.toThrow()` - För att verifiera giltiga inputs

What kind(s) of Code Coverage is shown in your code coverage tool?
What does it mean?

Vår coverage tool (Jest med Istanbul/nyc) visar 4 typer av coverage:

1. Statement Coverage (Rad-täckning)

- **Definition:** Procentandel av kod-statements som exekverats
- **Exempel i Password:**

```
const trimmedPW = pw.trim() // Statement 1
this.#passwordHash = this.#simpleHash(trimmedPW) // Statement 2
```

- **Mätning:** Har varje rad kod körts minst en gång?
- **I vårt projekt:** ~100% för Correct version

2. Branch Coverage (Gren-täckning)

- **Definition:** Procentandel av villkors-grenar (if/else, switch) som testats
- **Exempel i Password:**

```
if (this.#isTooShort(trimmedPW)) { // Branch 1: true
    throw new Error(...)
} // Branch 2: false (implicit)
```

- **Mätning:** Har både true OCH false fall testats för varje if-sats?
- **I vårt projekt:** Täcker både när errors kastas och inte kastas

3. Function Coverage (Funktions-täckning)

- **Definition:** Procentandel av funktioner/metoder som anropats
- **Exempel i Password:**
 - `constructor()` ☒
 - `#simpleHash()` ☒
 - `#isTooShort()` ☒
 - `#containsNumber()` ☒
 - `getPasswordHash()` ☒
 - `isPasswordSame()` ☒ (med test 3 och 11)
- **Mätning:** Har varje funktion anropats minst en gång?

4. Line Coverage (Linje-täckning)

- **Definition:** Likt Statement Coverage, men räknar fysiska rader
- **Skillnad:** Multi-statement lines kan räknas olika
- **Exempel:**

```
if (x) return y // En rad men två statements
```

Vad betyder 100% coverage?

Betyder INTE:

- **✗** Att koden är bug-fri
- **✗** Att alla edge cases är testade
- **✗** Att testerna är bra

Betyder:

- ☒ All kod har körts under test
- ☒ Inga döda kod-segment
- ☒ Alla grenar har testats

I vårt Password-projekt

Uppnår ~100% för:

- **Statements:** Alla rader körs
- **Branches:** Både error-cases och valid cases
- **Functions:** Alla metoder anropas
- **Lines:** Alla rader täcks

Men man kunde fortfarande missa buggar om man inte hade rätt assertions!

Why should a single test only have one assert/expect?

Huvudregel: Ett test = Ett koncept/beteende

Fördelar med single assert

1. Tydlighet och läsbarhet

```
// BRA - Ett assert, tydligt vad som testas
it('should trim spaces before hashing password', () => {
  const pwWithSpaces = new Password('  validPassword1  ')
  const pwWithoutSpaces = new Password('validPassword1')
  expect(pwWithSpaces.getPasswordHash()).toBe(pwWithoutSpaces.getPasswordHash())
})

// SÄMRE - Multiple asserts, otydligt fokus
it('should handle passwords correctly', () => {
  const pw = new Password('validPassword1')
  expect(pw.getPasswordHash()).toBeDefined()
  expect(pw.getPasswordHash()).not.toBe('validPassword1')
  expect(pw.getPasswordHash()).toBeGreaterThan(0)
})
```

2. Failure Pinpointing

Om ett test med flera asserts failar, får du bara se första felet:

```
it('multiple checks', () => {
  expect(result).toBe(5) // ✗ Failar här
  expect(result).toBeEven() // Körs aldrig!
  expect(result).toBePositive() // Körs aldrig!
})
```

Jag vet inte om de andra assertions också skulle faila.

3. Test Independence

Varje test bör testa EN sak:

```
// BRA - Tester separata koncept
it('should reject password without numbers', () => {
  expect(() => new Password('noNumbers')).toThrow('No number found')
})

it('should reject too short password', () => {
  expect(() => new Password('short1')).toThrow('Too short password')
})
```

4. Better Test Names

Ett assert = lättare att namnge testet beskrivande

När är multiple asserts OK?

1. Guard Assertions

```
it('should create valid password object', () => {
  const pw = new Password('validPassword1')
  expect(pw).toBeDefined() // Guard assertion
  expect(pw.getPasswordHash()).not.toBe('validPassword1') // Actual test
})
```

2. Testing Same Concept

```
it('should not have hash collisions', () => {
  const passwords = ['pwd1', 'pwd2', 'pwd3']
  const hashes = passwords.map(pw => new Password(pw).getPasswordHash())

  // Flera asserts men testar samma koncept (unikhet)
  expect(hashes[0]).not.toBe(hashes[1])
  expect(hashes[1]).not.toBe(hashes[2])
  expect(hashes[0]).not.toBe(hashes[2])
})
```

3. Arrange-Act-Assert pattern med state

```
it('should maintain state correctly', () => {
  const pw1 = new Password('validPassword1')
  const pw2 = new Password('validPassword1')

  expect(pw1.getPasswordHash()).toBe(pw2.getPasswordHash()) // Same input =
```



```
same hash
    expect(pw1.isPasswordSame(pw2)).toBe(true) // Related assertion, same concept
  })
```

I Password-projekt

Följer mostly single assert:

- **Test 1:** En assertion (hash $\neq$ plain text)
- **Test 2:** En assertion (trim-beteende)
- **Test 3-8:** En assertion var
- **Test 9:** Tekniskt flera map/set operationer men ett koncept (no collisions)

Slutsats: Single assert är en stark guideline för fokuserade, underhållbara tester, men kan brytas när det är logiskt motivated.

Sammanfattning

Dessa reflektioner visar att:

1. Optimerad testsuite med 9 tester täcker alla 10 buggar
2. Bra testdata fokuserar på edge cases och boundaries
3. Testa genom public API, inte private methods
4. 100% coverage är ett mål men inte alltid möjligt
5. Jest erbjuder många assertion types
6. Coverage mäts i 4 dimensioner (Statement, Branch, Function, Line)
7. Single assert per test ger tydlighet och bättre debugging